

CompTIA SecAI+

Lab 1.3 — Examine RAG Solutions

Using GPT OSS 20B in Open WebUI (with Gemini for retrieval-heavy steps)

Lab Setup

Retrieval-Augmented Generation (RAG) combines the reasoning of a language model with external knowledge. You upload documents into a Knowledge store, and the model pulls in the relevant pieces at query time instead of relying only on its training data. In this lab you will see how full-document retrieval, chunk size, chunk overlap, and Top-K affect the quality and accuracy of the answers.

Download the files from Labfiles 1.3 (development.md, api-endpoints.md, and rag.md) and save them in home/kali/Labfiles 1.3.

Before you begin, open Chat Controls (the sliders icon at the top-right of the chat), select Advanced Parameters, and apply these settings:

Parameter	Value
Max Tokens	2048
Every other parameter	Default (do not change)

Note: Do not change num_thread, num_batch, top_k, min_p, or any other advanced parameter — only Max Tokens should be changed. The others are not supported and cause a “property ... is unsupported” error.

Note: The free tier allows about 8,000 tokens per minute, and a chat resends its whole history on every message. Start a New Chat for each exercise, and if you see a “request too large / tokens per minute” message, start a New Chat (or wait a minute) and continue.

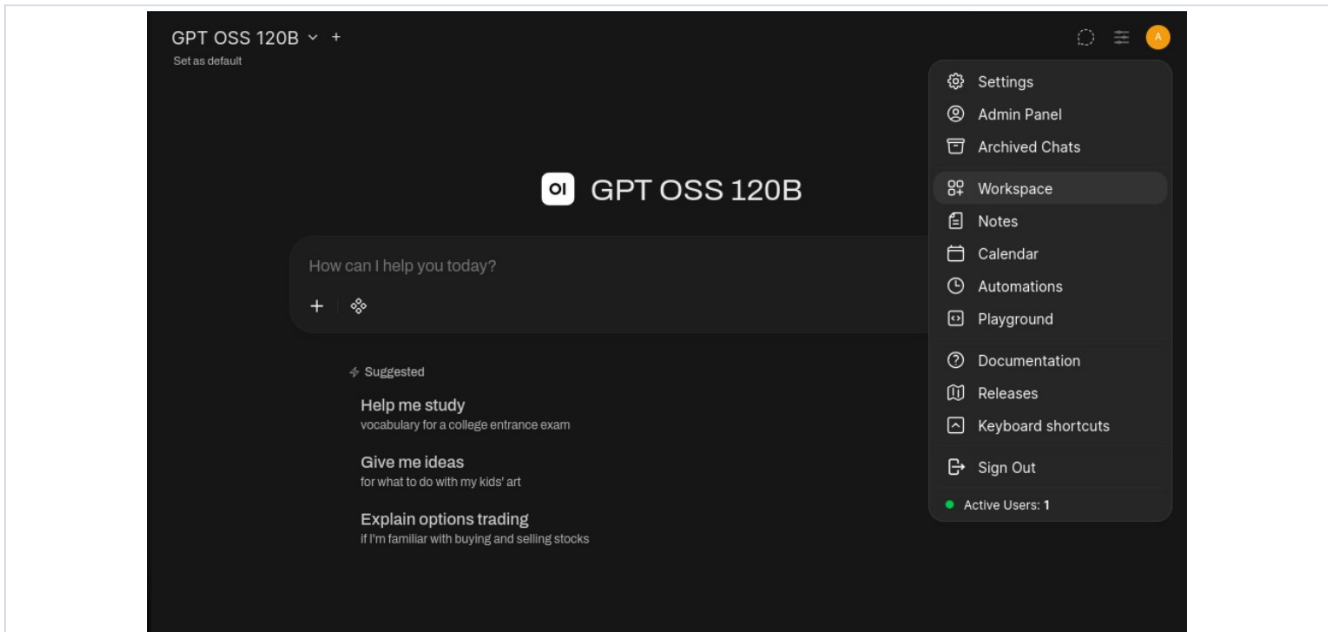
Note: The main model for this lab is GPT OSS 20B (Direct tab). Some retrieval-heavy steps work better on a larger-context model — where noted, you can switch to models/gemini-2.5-flash.

1.3.1 Examine RAG Solutions — Full-Document Retrieval

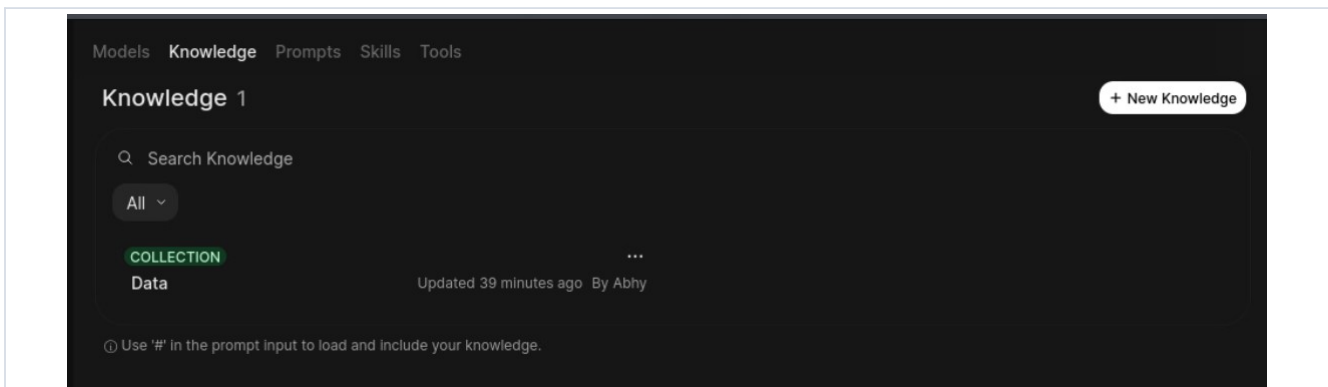
First you will load documents into a Knowledge store and query them with full-document retrieval, where the entire file is sent into the prompt.

Step 1: From the desktop, open Firefox, click the SecAI+ bookmark, and sign into Open WebUI.

Step 2: Open the **Workspace** (from the profile menu or the left menu), then select the **Knowledge** tab.



Step 3: Open the **Data** collection. The first time, click **+ New Knowledge** and upload the files.



Note: Upload development.md and api-endpoints.md from home/kali/Labfiles 1.3 (rag.md is added in the next step).

Step 4: From the Data collection, select **+** on the right, then **Upload files**.

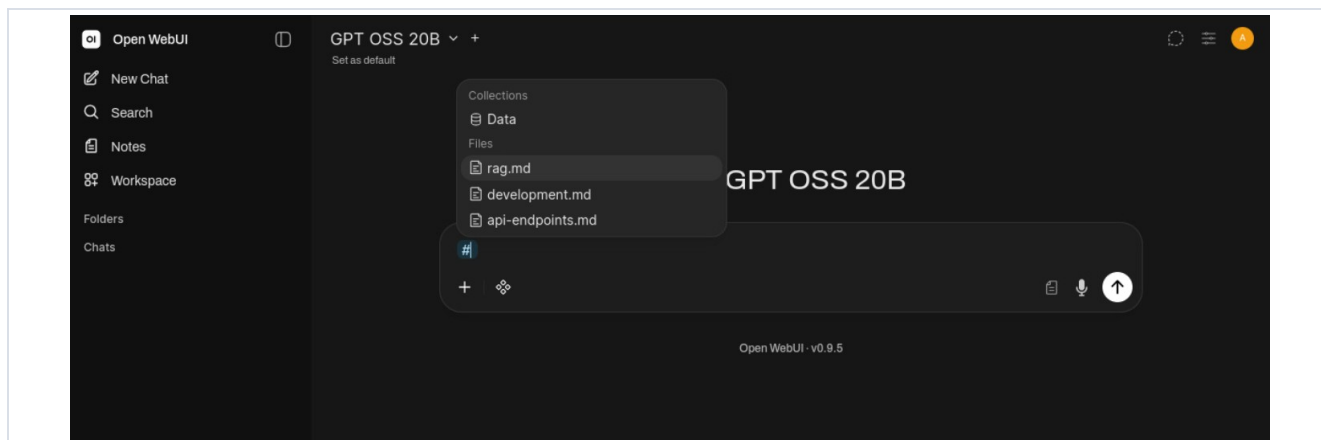
Step 5: Select **rag.md** from Labfiles 1.3 and choose Open. There should now be three files in the collection.

Step 6: Select **rag.md** from the collection to view the extracted text.

Insights: This is the text as the model sees it after extraction. With full-document retrieval, everything here is sent into the prompt at once — more tokens, more context consumed, and more chance the model focuses on irrelevant parts.

Step 7: Select **New Chat** from the top-left.

Step 8: In a new chat with **GPT OSS 20B**, enter **#** to bring up the knowledge store and select **rag.md**.



Step 9: Enter the following prompt and note the response:

Prompt 1:

What is the maximum default context length for Ollama?

Step 10: In the same chat, enter the following prompt and note the response:

Prompt 2:

Can this be increased?

Insights: Each response shows the sources used to generate it. The rag.md tag below each answer confirms it is the data source being used.

Step 11: Return to the Data collection and open rag.md to compare the results with the model's response.

Step 12: Start a New Chat, enter **#**, and select the **Data** collection (the whole collection).

Insights: Prompting against the whole collection lets you ask broad, comparative questions across multiple documents. Querying a single file keeps the scope narrow and precise. Together, these modes let you zoom out for trends or zoom in for detail.

Step 13: Enter the following prompt and note the response:

Prompt 3:

What are the benefits of using RAG?

Insights: Full-document retrieval quickly consumes the model's context window, especially across multiple files — and on the free tier it can also hit the per-minute token limit. Chunking (next section) keeps only the most relevant parts, so responses stay accurate and efficient.

1.3.2 Chunking, Overlap, and Retrieval Settings

Now you will break the documents into smaller chunks for retrieval, and adjust chunk size, overlap, and Top-K to change the balance between precision and completeness.

Step 1: From Open WebUI, select **User** (bottom-left), then **Settings**.

Step 2: Select **Admin Settings**, then **Documents** from the left menu.

Step 3: Under General, toggle **Bypass Embedding and Retrieval** OFF, then select Save.



Insights: This disables full-document retrieval and enables the embedding and chunking settings below.

Step 4: Start a New Chat, enter #, and select the Data collection.

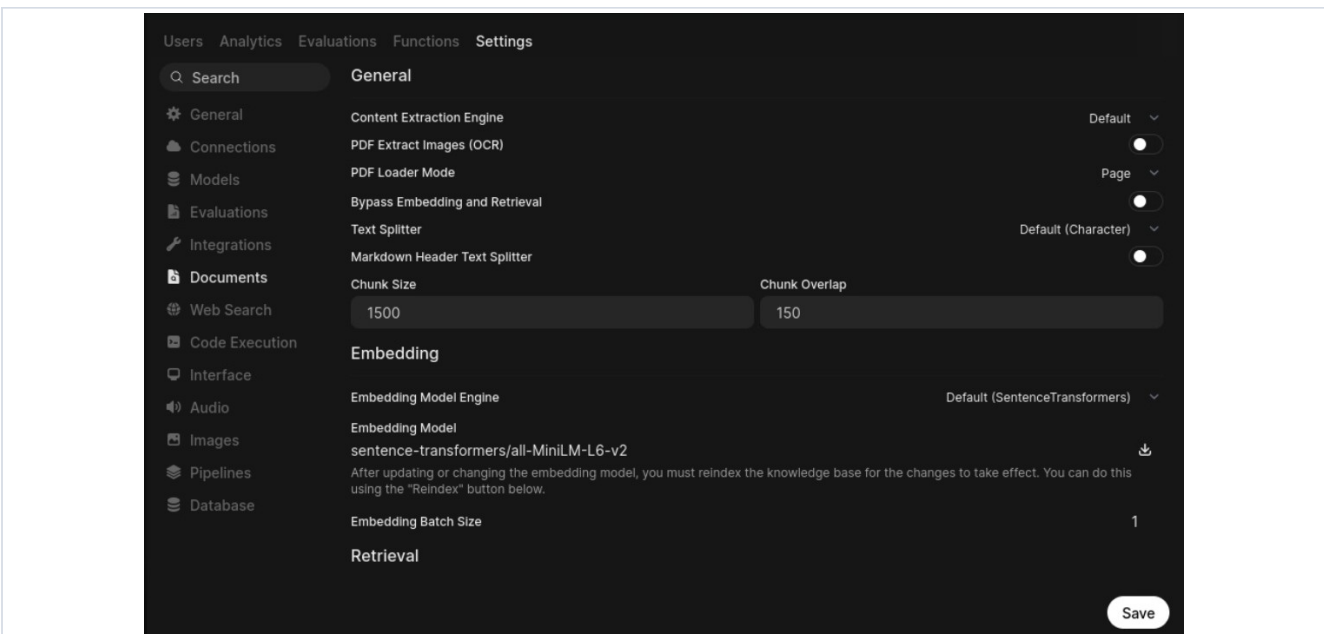
Step 5: Enter the following prompt and note the response:

Prompt 1:

What are the benefits of using RAG?

Insights: With the default settings (Chunk Size 1000, Chunk Overlap 100), the model may still fail to return the information we're looking for, even though it is present in the files.

Step 6: Return to Admin Settings → Documents.



Step 7: Change the **Chunk Size** to 300 and the **Chunk Overlap** to 80, then Save.

Insights: Chunk size controls how much text goes into each piece: larger values give more context but risk irrelevant detail; smaller values are more precise but fragment ideas. Overlap keeps continuity between chunks.

Step 8: Return to the Data collection (Workspace → Knowledge → Data).

Step 9: Remove rag.md, development.md, and api-endpoints.md (hover over each file and select the ✕ icon).

Step 10: Re-upload all three files to the collection.

Insights: Chunk size and overlap are applied at upload. Once a file is uploaded its chunks are fixed, so the files must be deleted and re-uploaded for new chunking settings to take effect.

Step 11: Start a New Chat, enter #, and select the Data collection.

Step 12: Enter the following prompt and note the response:

Prompt 2:

What are the benefits of using RAG?

Insights: With the new chunking settings, the responses should now be more accurate. Different file types often benefit from different chunking configurations, so some experimentation is normal.

Step 13: Start a New Chat, enter #, and select rag.md.

Step 14: Enter the following prompt and note the response:

Prompt 3:

List the steps under ## Detailed Instructions.

Insights: With small chunks the model will likely have trouble finding the mentioned section.

Step 15: Return to Admin Settings → Documents.

Step 16: Under **Retrieval**, set the **Top K** value to 10, then Save.

Insights: Top-K is a retrieval setting, so files do not need to be re-uploaded when it changes. It controls how many chunks are pulled at query time.

Step 17: Start a New Chat, enter #, and select rag.md.

Step 18: Enter the following prompt and note the response:

Prompt 4:

List the steps under ## Detailed Instructions.

Insights: A higher Top-K helps retrieval find the right section by pulling in more chunks, but because the chunk size is still very small (300), each chunk holds only a sliver of the text — so the answer is still incomplete.

Step 19: Return to Admin Settings → Documents and set the **Chunk Size** to 1500 and **Chunk Overlap** to 150, then Save.

Step 20: Return to the Data collection, delete rag.md, and re-upload it.

Step 21: Start a New Chat, enter #, and select rag.md.

Step 22: Enter the following prompt and note the response:

Prompt 5:

List the steps under ## Detailed Instructions.

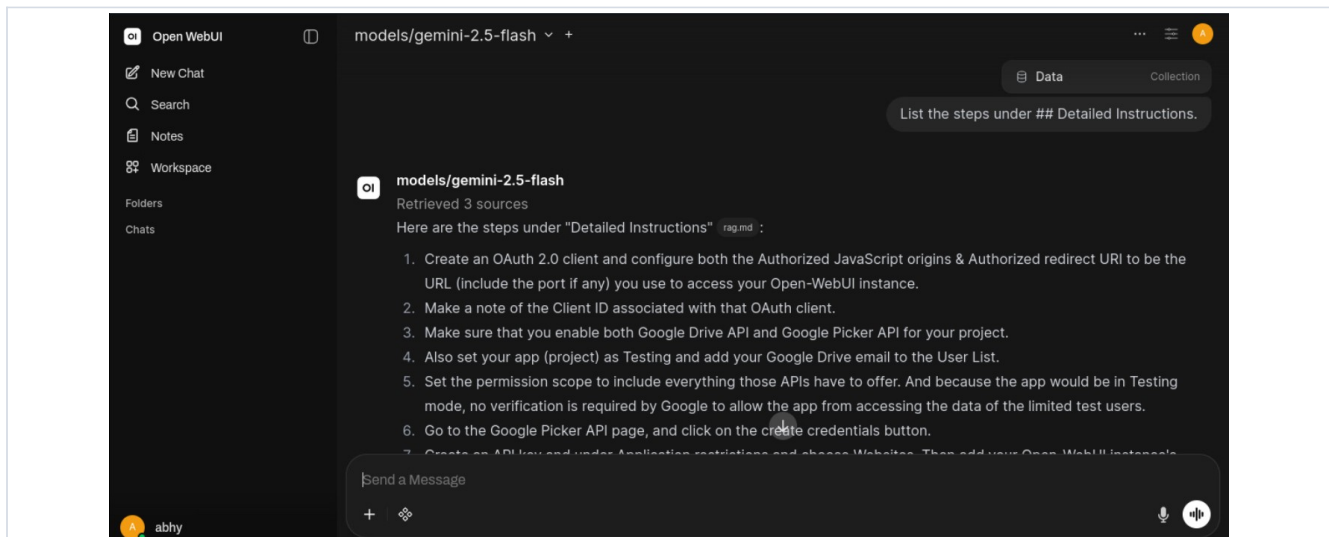
Insights: With the larger chunk size and overlap, the model now returns a complete answer when querying the single file. You can verify its accuracy by viewing rag.md in the Data collection.

Key Lesson — Single File vs the Whole Collection

You may have noticed that asking for the steps under **## Detailed Instructions** works when you query `rag.md` by itself, but not when you query the whole Data collection at once. This is an important RAG lesson, not a fault.

Insights: When you query the whole collection, the Top-K budget is shared across all three files, and chunks from `development.md` (which is also full of steps and instructions) compete for those slots. So the specific chunk holding **## Detailed Instructions** can get crowded out, and the model reports it cannot find the section. Querying `rag.md` alone devotes the whole Top-K budget to that one file, so the section is retrieved reliably.

Note: On GPT OSS 20B (free tier) this is made harder by the per-minute token limit: raising Top-K high enough to pull the section out of the whole collection can push the request over the 8,000-token limit. For a precise lookup, query the single file. If you want to retrieve a specific section from the whole collection, use a larger-context model such as `models/gemini-2.5-flash` — as shown below, it retrieves the section successfully from the Data collection.



Insights: Match the scope to the question: query a single file for a precise lookup, use the whole collection for broad questions, and switch to a larger-context model when you need whole-collection precision.

Troubleshooting

Common messages and situations in this lab, and what to do:

Situation	Do this
Whole-collection query can't find a specific section	Expected on a small model. Query the single file (e.g. rag.md) instead, raise Top-K and Chunk Size, or switch to models/gemini-2.5-flash for whole-collection retrieval.
Changing Chunk Size / Overlap doesn't change the answer	Chunking happens at upload. Delete the files and re-upload them for new chunk settings to take effect. (Top-K changes do not need a re-upload.)
"Request too large ... tokens per minute (TPM): Limit 8000"	The retrieved chunks plus the prompt are over the per-minute limit. Query a single file, lower Top-K or Chunk Size, start a New Chat, or use models/gemini-2.5-flash.
"property 'num_batch' / 'top_k' / ... is unsupported"	You customised a parameter that isn't supported. Open Chat Controls → Advanced Parameters and set it back to Default. Only Max Tokens should be changed.